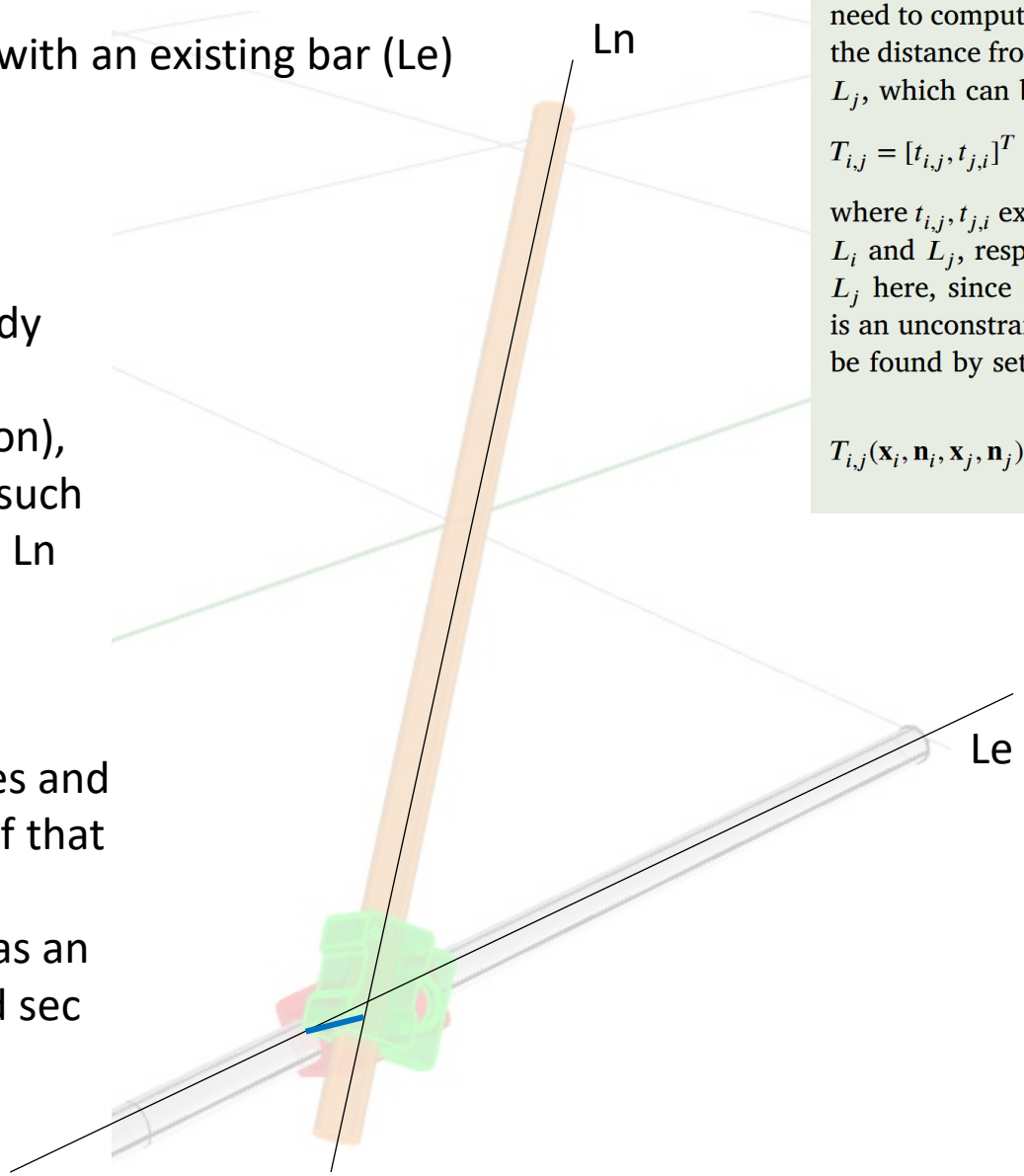
S1:
Add one new bar (Ln) to be joined with an existing bar (Le)
T1:
Bar axis line generation for Ln

We assume that the designer already initialize Ln's line to be at the right orientation (vector pointing direction), we only need the script to move it such that the shortest distance between Ln and Le equals to a preset value D.

This only needs us to compute the shortest segment between two lines and then move Ln along the direction of that segment. This shortest segment between two infinitely long lines has an analytical solution. [refer code] and sec 3.5 of the CADJ paper



To compute the distance between a pair of clamps on a bar, we will need to compute the arc-length parameter $t_{i,j}$. This parameter expresses the distance from the bar end point $\mathbf{x}_i$ of bar L_i to the its connector with L_j, which can be computed by:

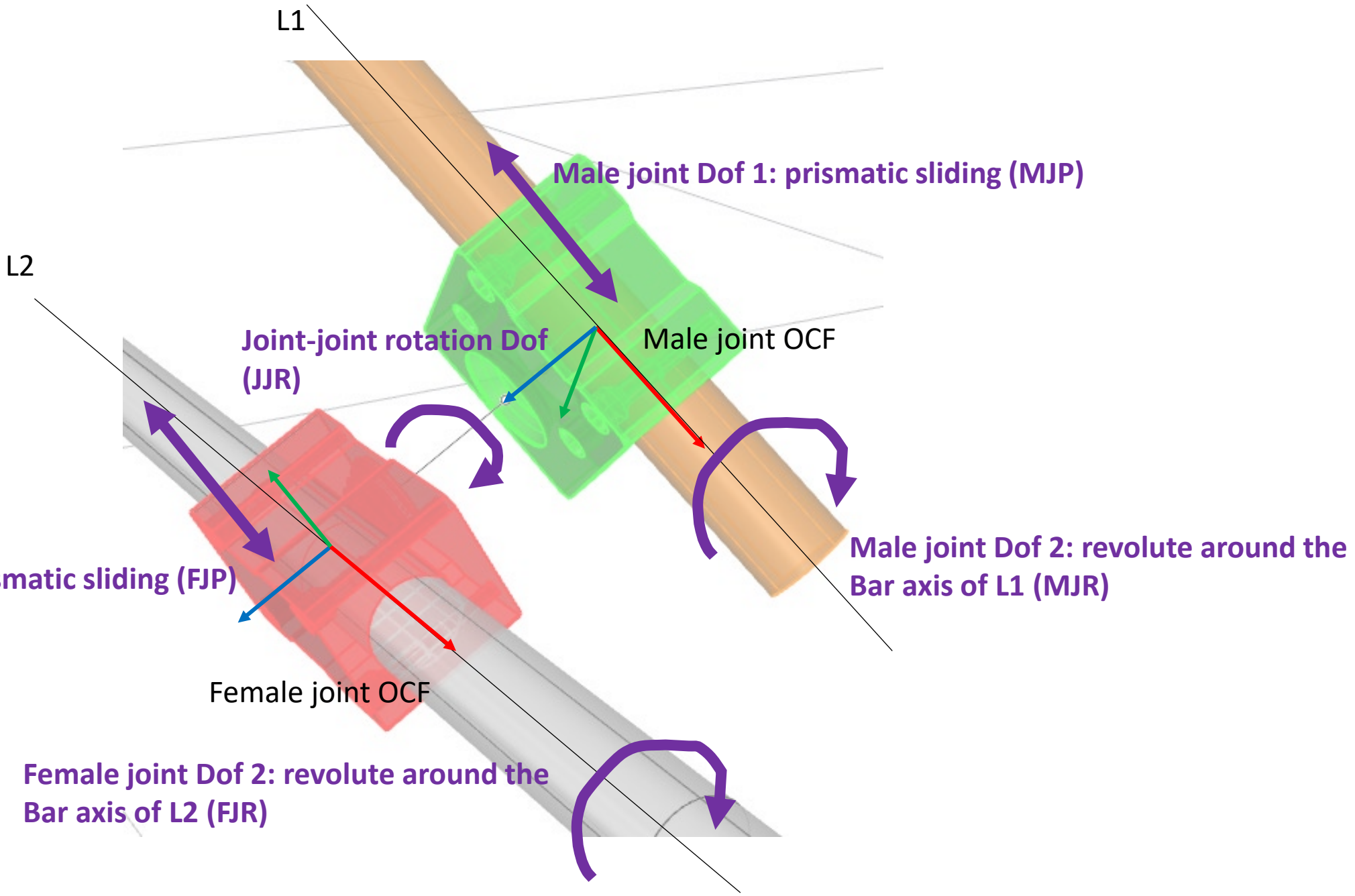
$$T_{i,j} = [t_{i,j}, t_{j,i}]^T = \operatorname{argmin} \|(\mathbf{x}_i + t_{i,j}\mathbf{n}_i) - (\mathbf{x}_j + t_{j,i}\mathbf{n}_j)\|^2$$

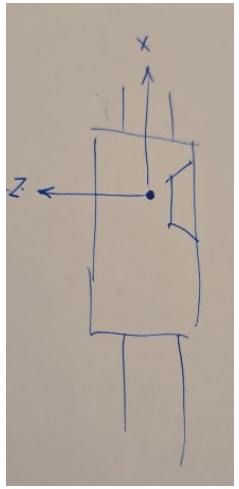
where $t_{i,j}, t_{j,i}$ express the same connector's arc-length parameters on bar L_i and L_j, respectively. We only consider non-parallel pairs of L_i and L_j here, since in the parallel case we are free to choose t. Since this is an unconstrained, quadratic optimization problem, the optimum can be found by setting the gradient with respect to t to zero:

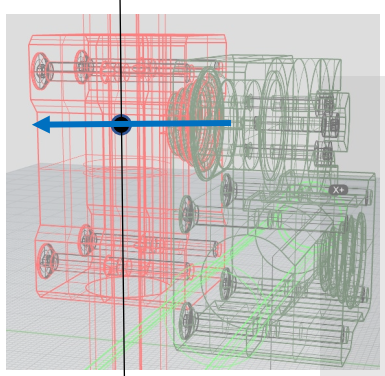
$$T_{i,j}(\mathbf{x}_i, \mathbf{n}_i, \mathbf{x}_j, \mathbf{n}_j) = \begin{bmatrix} \mathbf{n}_i^T\mathbf{n}_i & -\mathbf{n}_i^T\mathbf{n}_j \\ -\mathbf{n}_i^T\mathbf{n}_j & \mathbf{n}_j^T\mathbf{n}_j \end{bmatrix}^{-1} \begin{bmatrix} (\mathbf{x}_j - \mathbf{x}_i)^T\mathbf{n}_i \\ (\mathbf{x}_i - \mathbf{x}_j)^T\mathbf{n}_j \end{bmatrix} \tag{17}$$

Here we express a line with an anchor point x and a normal n. The shortest segment's two end points can be found by optimizing for the t parameter to locate them on both lines.

Before we move on to S1-T2, we can view our joints as rigid body links of a multi-dof kinematic robots. Here are the convention of each link's object frame, as well as movable joints.
A more detailed OCF convetion is given in the next slide.

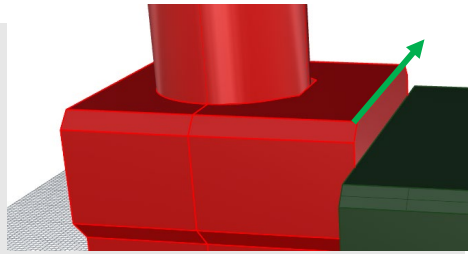


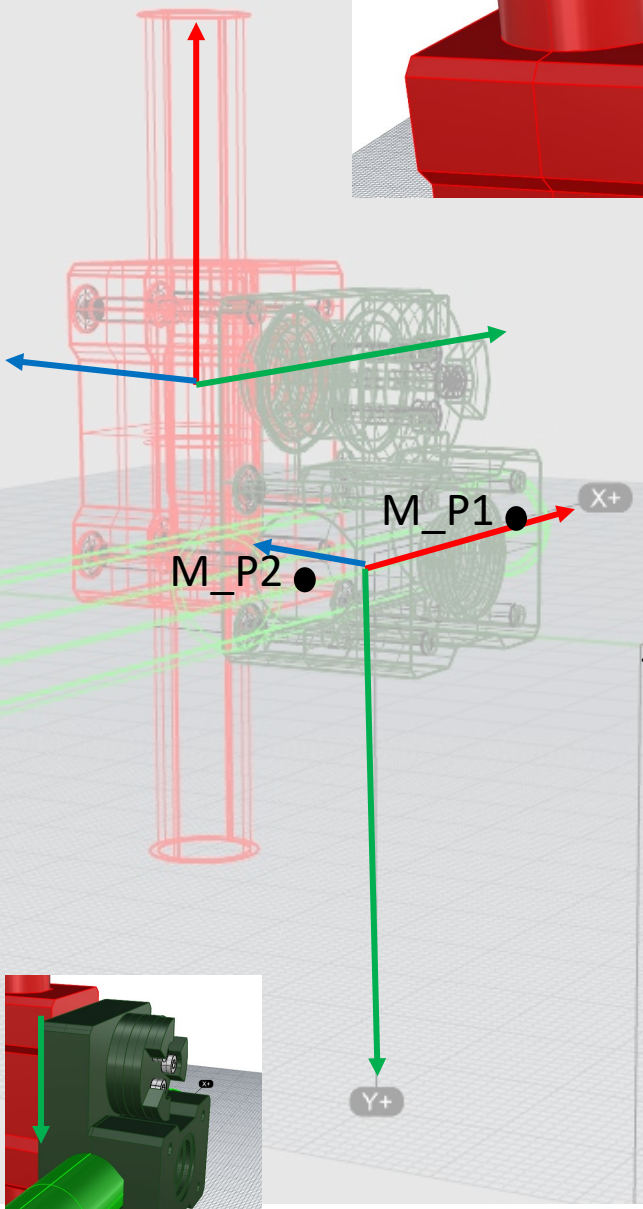




OCF convention
for **female joint**:

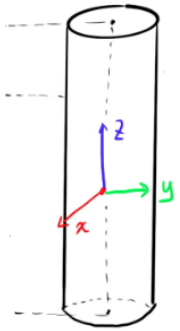
- Origin is the center of the male joint screw hole projected on the line. (see inset on top-left)
- X axis align with bar axis
- Y axis parallel to the edge of the brep (see inset top-right)
- Z axis pointing **the same way as** the male joint screwing direction

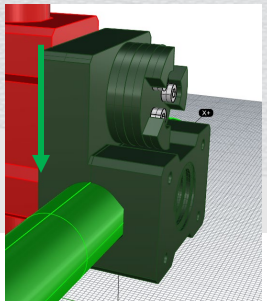




OCF convention
for **male joint**:

- Origin at the midpoint between M_P1 and M_P2
- X axis align with bar axis
- Y axis parallel to the edge of the brep (see inset)
- Z axis pointing to the screw insertion direction





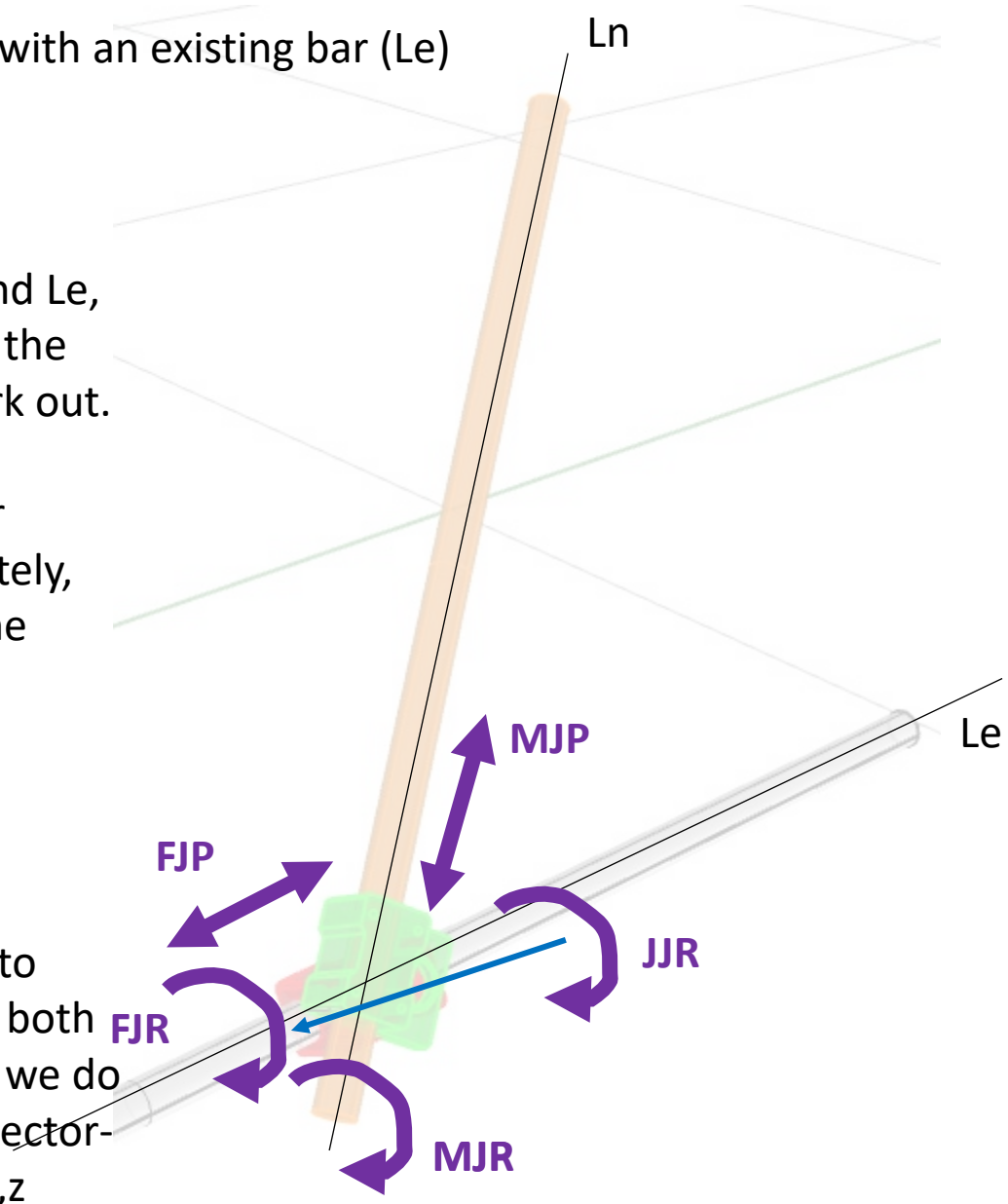
The OCF for a bar line axis is the mid point of the line, with z axis along the longitudal axis, and x-y can be set freely with a rotation around the z.

S1:
Add one new bar (Ln) to be joined with an existing bar (Le)
T2:
Joint placement

After S1-T1, we lock the lines Ln and Le, now we optimize 5 dofs (purple in the image), so that the kinematics work out. One way to formulate this is:
We do Forward Kinematics (FK) for female joint and male joint separately, and then ”make things meet” in the middle:

Min_{fjp, fjr, mjp, mjr, jjr}
|| FK(fjp, fjr, jjr) – FK(mjp, mjr) ||

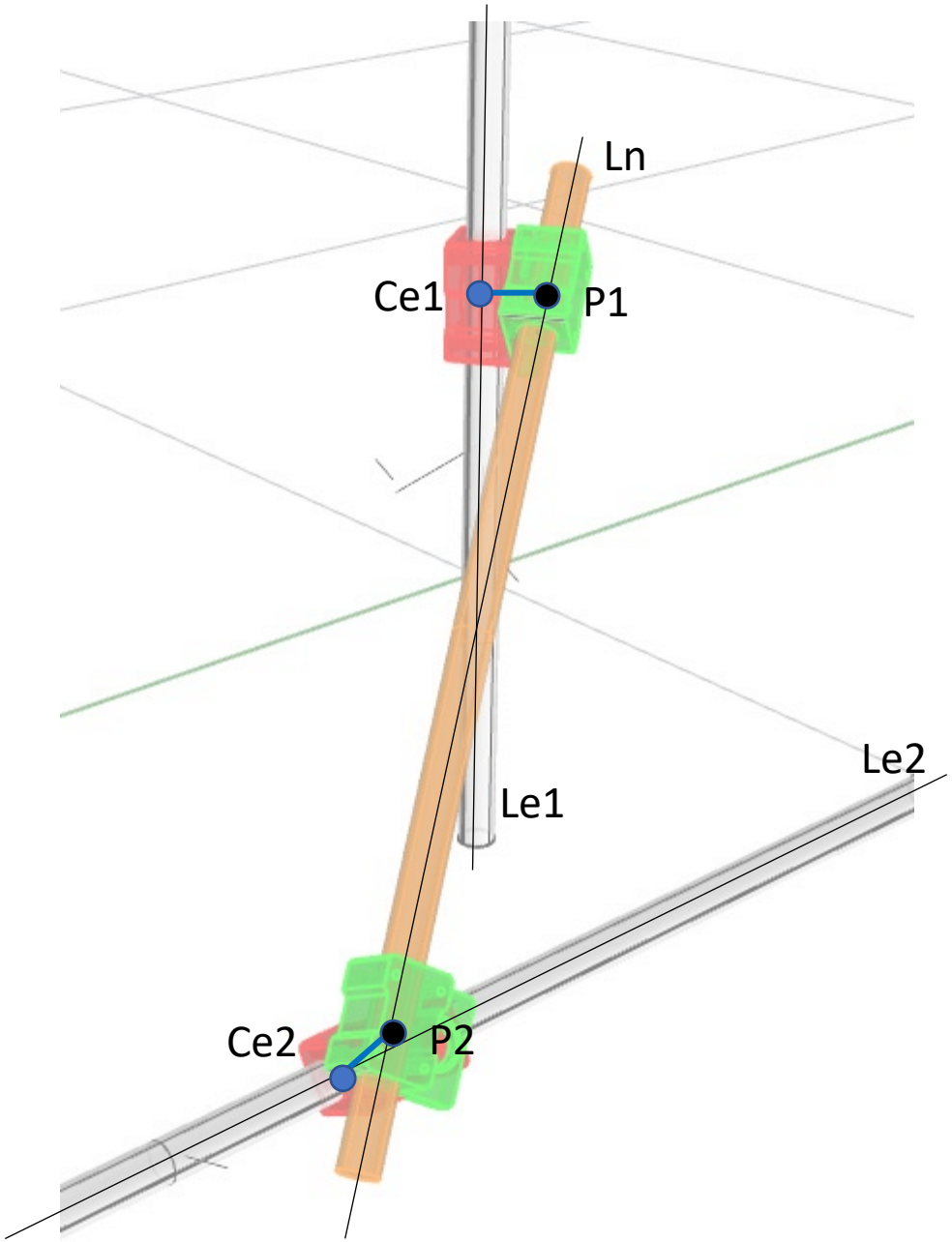
Note that the || - || norm here is to compare two frames that includes both position and orientation. I suggest we do a sum of original L2 distance and vector-vector distance for each pair of x,y,z axes.



S2:
Add one new bar (Ln) to be joined with TWO existing bar (Le1, Le2)
T1:
Bar axis line generation for Ln

Here the designer inputs four things:
- Lines Le1 and Le2
- The anchor point of the “contact normals” to Ln

Here the contact normal are the line sgements that forms the shortest distance between Ln and the two lines.
Our goal is to solve for an anchor point xn and a direction nn to define Ln, such that:
- The shortest segments between Ln-Le1 have one end point to be the given Ce1
- The shortest segments between Ln-Le2 have one end point to be the given Ce2
- These two segements’ length equals to a pre-given value D



S2:
Add one new bar (Ln) to be joined with TWO existing bar (Le1, Le2)
T1:
Bar axis line generation for Ln
CONTINUED

I am not very sure how we would formulate the optimization problem in the previous page, but I think the following equaitons will help, one gives us the anlayitcal expression of the shortest distance between two lines, and the second helps us to find the exact location of the contact point. If possible, I would prefer a analytical solution, but otherwise some optimization formulation that uses scipy (not these paid optimizer like knitro or gubori) also works for me.

Mathematically, the central axis of an infinite-length bar can be represented by a point $\mathbf{x}_i$ on the line and a vector $\mathbf{n}_i$. When not in parallel, the distance between two such bars can be simplified as an analytical expression:

$$d_\infty[L_i, L_j] = (\mathbf{n}_i/\|\mathbf{n}_i\| \times \mathbf{n}_j/\|\mathbf{n}_j\|)^T(\mathbf{x}_i - \mathbf{x}_j) \tag{3}$$

$$T_{i,j} = [t_{i,j}, t_{j,i}]^T = \operatorname{argmin} \|(\mathbf{x}_i + t_{i,j}\mathbf{n}_i) - (\mathbf{x}_j + t_{j,i}\mathbf{n}_j)\|^2$$

where $t_{i,j}, t_{j,i}$ express the same connector's arc-length parameters on bar L_i and L_j, respectively. We only consider non-parallel pairs of L_i and L_j here, since in the parallel case we are free to choose t. Since this is an unconstrained, quadratic optimization problem, the optimum can be found by setting the gradient with respect to t to zero:

$$T_{i,j}(\mathbf{x}_i, \mathbf{n}_i, \mathbf{x}_j, \mathbf{n}_j) = \begin{bmatrix} \mathbf{n}_i^T\mathbf{n}_i & -\mathbf{n}_i^T\mathbf{n}_j \\ -\mathbf{n}_i^T\mathbf{n}_j & \mathbf{n}_j^T\mathbf{n}_j \end{bmatrix}^{-1} \begin{bmatrix} (\mathbf{x}_j - \mathbf{x}_i)^T\mathbf{n}_i \\ (\mathbf{x}_i - \mathbf{x}_j)^T\mathbf{n}_j \end{bmatrix} \tag{17}$$

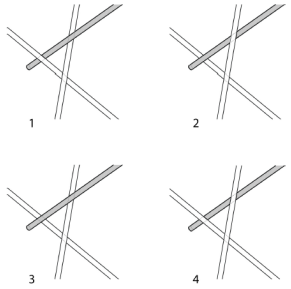
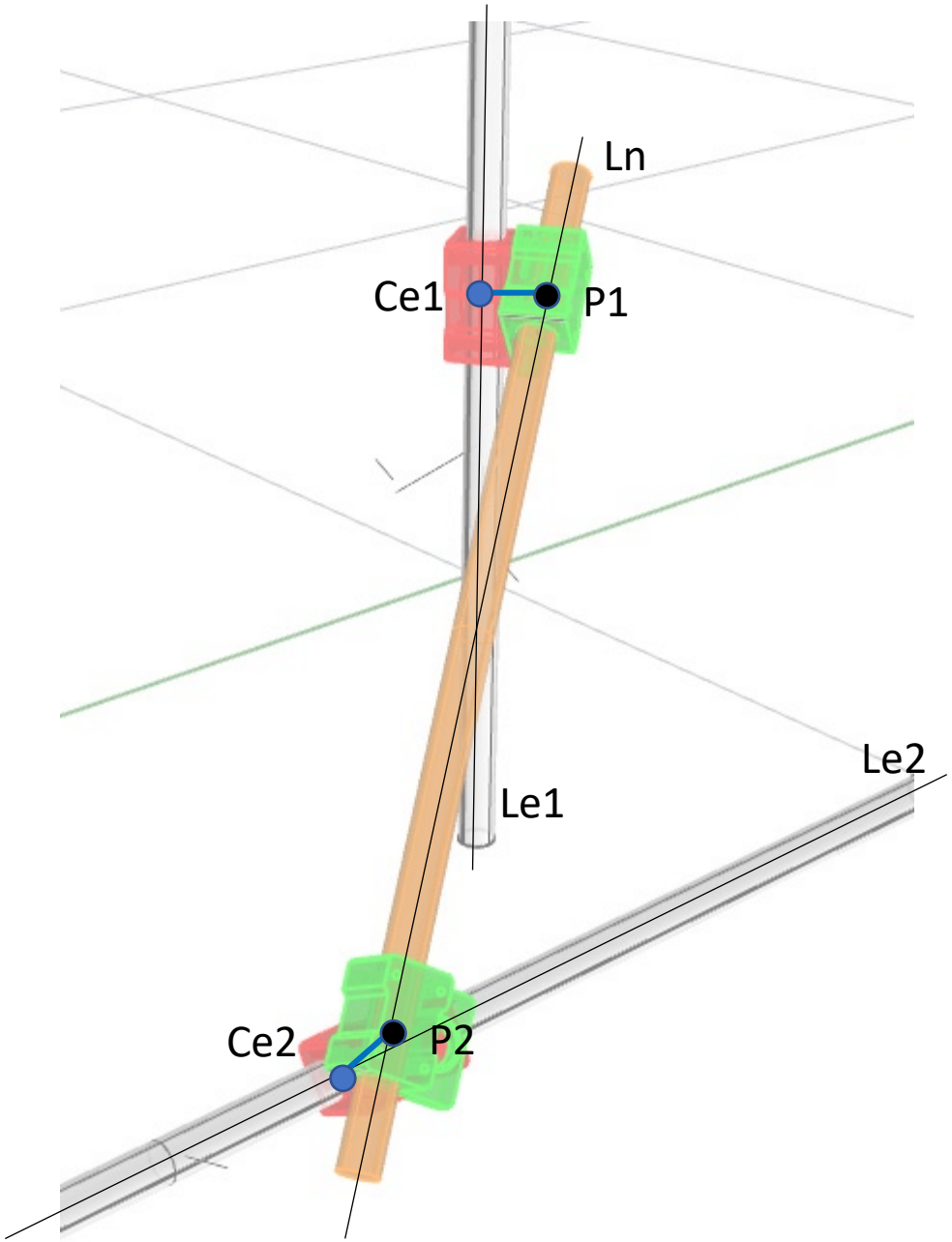
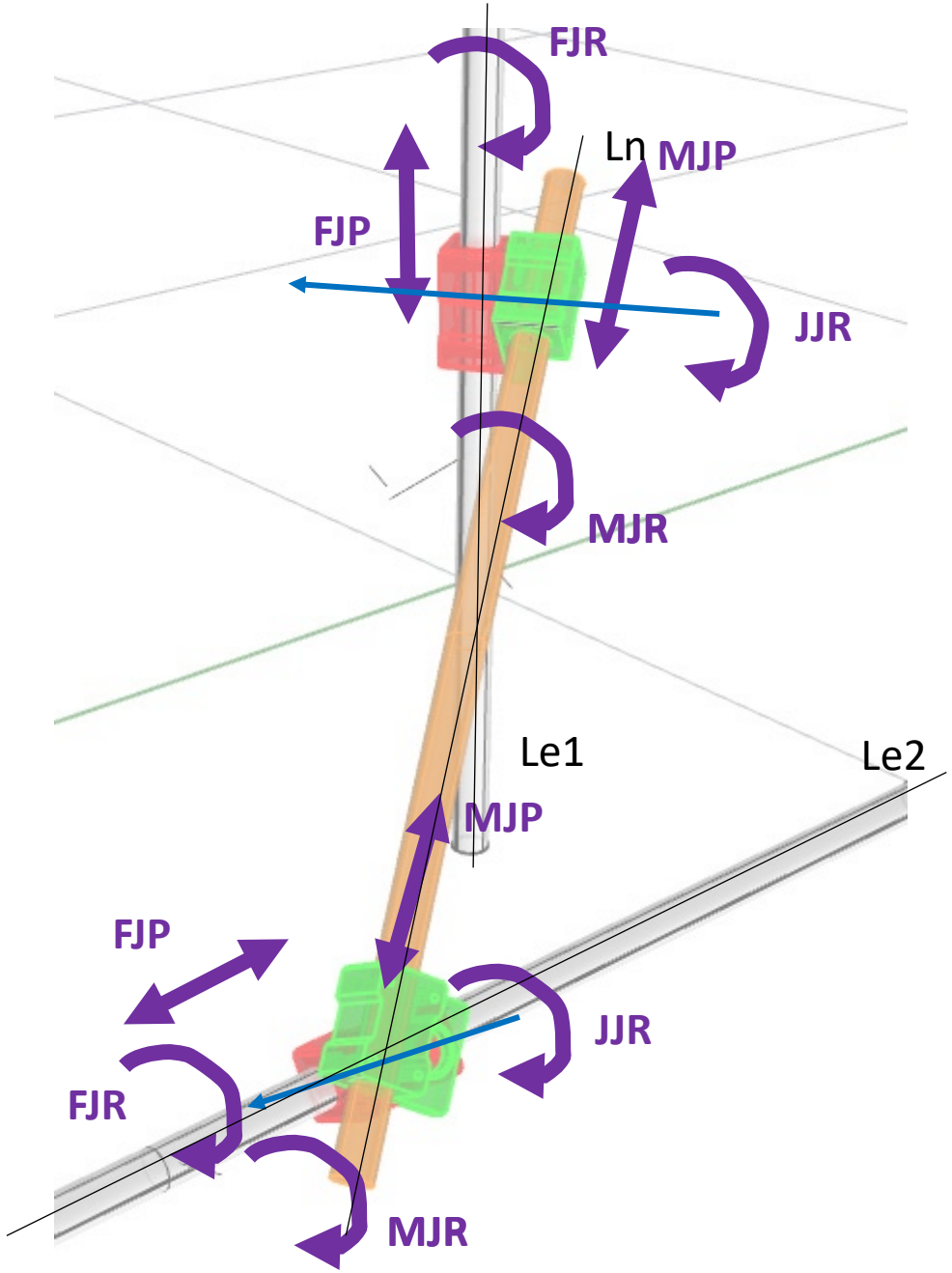


Fig. 3.5: Potential connection options for one bar to two existing bars. Img.: SP.



S2:
Add one new bar (Ln) to be joined with TWO existing bar (Le1, Le2)
T2:
Joint placement

Here we also lock the line Ln generated from S2-T2. Then the joint placement is simply run the FK-based joint optimization from S1-T2 twice for each end. Reuse that as much as possible.



Implementation details

UI

- We want two scripts for T1 and T2. In the end, the user will call this scripts thru buttons on the Rhion UI bar (via Run ScriptEditor <point to python script path>). Not so sure if we need four buttons for all four scenarios, but let's minimize number of buttons.
- In Rhino, left click the button will prompt the user to click lines or points as inputs. Right click will run the script again with the inputs from the last call.

Output

- For T1, the script simply generate a line and back it back into Rhino, you can make a default length there that is long enough but not crazy, and user will resize according to their need.
- For T2, we will have some Block definitions with the name "FemaleJoint" and "MaleJoint" placed at the world origin according to the OCF convention. The T2 script should directly generate a Block instance, with additional JSON format type of info saved in the "Attribute User text", which I could parse easily in Python for robot planning etc.